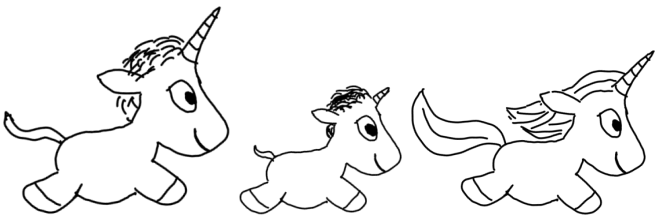
Das Dreieinhornskript zur

Vorlesung
Informatik III – Theoretische Informatik

Formale Sprachen und Berechenbarkeit



Peter Thiemann

thiemann@informatik.uni-freiburg.de

SS 2026

Uni Freiburg

Zuletzt aktualisiert: 2026-04-21

Zur Geschichte dieses Skriptes

Das Skript wurde von Ralph Lesch begonnen und war zunächst ein Vorlesungsmitschrieb der im WS 2015/16 von Prof. Dr. Peter Thiemann gehaltenen Vorlesung.
`https://proglang.informatik.uni-freiburg.de/teaching/info3/2015ws/`

Für die im WS 2016/17 von Prof. Dr. Peter Thiemann gehaltene Vorlesung wurde das Skript von diesem und Luminous Fennell überarbeitet.
`https://proglang.informatik.uni-freiburg.de/teaching/info3/2016ws/`

Für die im WS 2017/18 von Dr. Matthias Heizmann gehalten Vorlesung wurde das Skript von diesem und Christian Schilling überarbeitet.
`http://swt.informatik.uni-freiburg.de/teaching/WS2017-18/info3`

Im WS 2018/19 wurde die Vorlesung erneut von Dr. Matthias Heizmann gehalten und das Skript nochmals überarbeitet.
`http://swt.informatik.uni-freiburg.de/teaching/WS2018-19/info3`

Im SS 2026 wurde die Vorlesung von Prof. Dr. Peter Thiemann gehalten und das Skript wieder überarbeitet.
`https://proglang.github.io/teaching/26ss/ti.html`

Trotz der vielen Bearbeitungsschritte ist dieses Skript noch nicht "fertig". Es soll kein eigenständiges Werk sondern eine Ergänzung zu den Vorlesungen sein. Die Vorlesungen wurden großteils an der Tafel gehalten. Vereinzelt eingesetzte Folien sowie Übungsaufgaben und Klausuren sind auf den jeweiligen Vorlesungswebseiten verlinkt.

Der Latex Quellcode des Skriptes ist in einem Git Repositorium verfügbar.
`https://github.com/proglang/Informatik-III-Theoretische-Informatik`

1 Vorspann: Sprachen

Def. 1.1: Ein *Alphabet* ist eine nicht leere, endliche Menge von *Zeichen*. ◇

Zeichen sind hier beliebige abstrakte Symbole.

Bsp.: für Alphabete, die in dieser Vorlesung, im täglichem Umgang mit Computern oder in der Forschung an unserem Lehrstuhl eine Rolle spielen

- $\{a, \ldots, z\}$
- $\{0, 1\}$
- $\{\mathit{rot}, \mathit{gelb}, \mathit{grün}\}$ (Ampelfarben)
- Die Menge aller ASCII-Symbole
- Die Menge aller Statements eines Computerprogramms

Wir verwenden typischerweise den griechischen Buchstaben Σ als Namen für ein Alphabet und die lateinischen Buchstaben a, b, c als Namen für Zeichen.Im Folgenden sei Σ immer ein beliebiges Alphabet.

Def. 1.2: Ein *Wort* ist eine endliche Folge von Elementen aus Σ. Die leere Folge nennen wir das *leere Wort* geschrieben als ε. Wir bezeichnen die Menge aller Wörter mit Σ^* und die Menge aller nicht leeren Wörter mit $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$.

Hier die induktive Definition.

1. $\varepsilon \in \Sigma^*$.
2. $a \in \Sigma$ und $w \in \Sigma^*$ impliziert $a.w \in \Sigma^*$.

◇

Hier einige Konventionen:

- Wir schreiben ein Wort immer ohne Trennsymbole wie z.B. Komma. Also `einhorn` statt `e,i,n,h,o,r,n`.
- Wir verwenden ε für das leere Wort.
- Wir verwenden typischerweise u, v, w als Namen für Wörter.

Bsp.: für Wörter über $\Sigma = \{a, \ldots, z\}$

- `rambo` (Länge 5)
- `eis`, `ies` (beide Länge 3 aber ungleich)
- ε (Länge 0)

Aus der induktiven Definition ergibt sich das folgende Beweisprinzip für Aussagen $\varphi(w)$ über Worte w:

$$\frac{\varphi(\varepsilon) \quad \forall a, w.\varphi(w) \Rightarrow \varphi(a.w)}{\forall w \in \Sigma^*.\varphi(w)}$$

Wörter lassen sich „verketten"/„aneinanderhängen". Die entsprechende Operation heißt *Konkatenation*, geschrieben „$\cdot$" (wie Multiplikation).

Def. 1.3 (Konkatenation von Wörtern)**:** Die *Konkatenation*, $\cdot : \Sigma^* \times \Sigma^* \to \Sigma^*$, wird für alle $v \in \Sigma^*$ induktiv definiert durch

1. $\varepsilon \cdot v = v$.
2. für alle $a \in \Sigma$ und $u \in \Sigma^*$: $(a.u) \cdot v = a.(u \cdot v)$.

$\diamond$

Eigenschaften der Konkatenation: $(\Sigma^*, \varepsilon, \cdot)$ ist ein Monoid:

1. $\varepsilon \cdot v = v$ (leeres Wort ist linksneutrales Element)
2. $v \cdot \varepsilon = v$ (leeres Wort ist rechtsneutrales Element)
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ (Assoziativität)

Beweis:

1. $\varepsilon \cdot v = v$ (Definition)
2. $v \cdot \varepsilon = v$ Induktion über v:
   - IA: $(v = \varepsilon)$ $\varepsilon \cdot \varepsilon = \varepsilon$ (Definition)
   - IS: $(v \rightsquigarrow a.v)$ $(a.v) \cdot \varepsilon = a.(v \cdot \varepsilon) \overset{IV}{=} a.v$
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ Induktion über u:
   - IA: $(u = \varepsilon)$ $\varepsilon \cdot (v \cdot w) = (v \cdot w) = (\varepsilon \cdot v) \cdot w$ (Definition)
   - IS: $(u \rightsquigarrow a.u)$ $(a.u) \cdot (v \cdot w) \overset{Def}{=} a.(u \cdot (v \cdot w)) \overset{IV}{=} a.((u \cdot v) \cdot w) \overset{Def}{=} ((a.u) \cdot v) \cdot w$

Bsp.:

- $\texttt{eis} \cdot \texttt{rambo} = \texttt{eisrambo}$
- $\texttt{rambo} \cdot \varepsilon = \texttt{rambo} = \varepsilon \cdot \texttt{rambo}$

Der Konkatenationsoperator „$\cdot$“ wird oft weggelassen (ähnlich wie der Multiplikationsoperator in der Arithmetik). Ebenso können durch die Assoziativität Klammern weggelassen werden.

$w_1 w_2 w_3$ steht also auch für $w_1 \cdot w_2 \cdot w_3$, für $(w_1 \cdot w_2) \cdot w_3$ und für $w_1 \cdot (w_2 \cdot w_3)$

Bemerkung: Die Zeichenfolge $\texttt{rambo}\varepsilon$ ist *kein* Wort. Diese Zeichenfolge ist lediglich eine Notation für eine Konkatenationsoperation, die ein Wort der Länge 5 (nämlich `rambo`) beschreibt.

Wörter lassen sich außerdem *potenzieren*:

Def. 1.4: Die *Potenzierung* von Wörtern, $\cdot^{\cdot} : \Sigma^* \times \mathbb{N} \to \Sigma^*$, ist induktiv definiert durch

1. $w^0 = \varepsilon$
2. $w^{n+1} = w \cdot w^n$ ◇

Bsp.: $\texttt{eis}^3 \overset{(2.)}{=} \texttt{eis}\cdot\texttt{eis}^2 \overset{\text{zweimal } (2.)}{=} \texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}^0 \overset{(1.)}{=} \texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}\cdot\varepsilon = \texttt{eiseiseis}$

Def. 1.5: Eine *Sprache* über Σ ist eine Menge $L \subseteq \Sigma^*$. Gleichwertig: $L \in \mathcal{P}(\Sigma^*)$. ◇

Bsp.:

- $\{\texttt{eis}, \texttt{rambo}\}$
- $\{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$
- $\{\}$ (die „leere Sprache“)
- $\{\varepsilon\}$ (ist verschieden von der leeren Sprache)
- Σ^*

Sämtliche Mengenoperationen sind auch Sprachoperationen, insbesondere Schnitt ($L_1 \cap L_2$), Vereinigung ($L_1 \cup L_2$), Differenz ($L_1 \setminus L_2$) und Komplement ($\overline{L_1} = \Sigma^* \setminus L_1$).

Weitere Operationen auf Sprachen sind Konkatenation und Potenzierung, sowie der *Kleene-Abschluss*.

Def. 1.6 (Konkatenation und Potenzierung von Sprachen)**:** Seien $U, V \subseteq \Sigma^*$. Dann ist die *Konkatenation* von U und V definiert durch

$$U \cdot V = \{uv \mid u \in U, v \in V\}$$

und die *Potenzierung* von U induktiv definiert durch

1. $U^0 = \{\varepsilon\}$
2. $U^{n+1} = U \cdot U^n$ ◇

Bsp.:

- $\{\texttt{eis}, \varepsilon\} \cdot \{\texttt{rambo}\} = \{\texttt{eisrambo}, \texttt{rambo}\}$
- $\{\texttt{eis}, \varepsilon\} \cdot \{\} = \{\}$
- $\{\}^0 = \{\varepsilon\}$
- $\{\}^4 = \{\}$
- $\{\texttt{eis}, \varepsilon\}^2 = \{\varepsilon, \texttt{eis}, \texttt{eiseis}\}$

Wie bei der Konkatenation von Wörtern dürfen wir den Konkatenationsoperator auch weglassen.

Def. 1.7 (Kleene-Abschluss, Kleene-Stern, Kleene-Plus)**:** Gegeben eine Sprache $U \subseteq \Sigma^*$, definieren wir den *Kleene-Abschluss* U^* wie folgt.

$$U^* = \bigcup_{n \in \mathbb{N}} U^n$$

Wie nennen den Operator $\cdot^* : \mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$, der für eine gegebene Sprache den Kleene-Abschluss liefert, den *Kleene-Stern*. Analog definieren wir den *Kleene-Plus* Operator $\cdot^+ : \mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$ als

$$U^+ = \bigcup_{n \geq 1} U^n.$$

◇

Es gilt also $U^* = U^+ \cup \{\varepsilon\}$.

Def. 1.8 (Alternative Definition für Σ^*)**:** Sei $n \in \mathbb{N}$.

- $\Sigma^n = \{1, \ldots, n\} \to \Sigma$ Worte der Länge n
- insbesondere $\Sigma^0 = \{\} \to \Sigma$ enthält nur ein Element, ε

- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$
- Konkatenation: $u \in \Sigma^n$, $v \in \Sigma^m \Rightarrow u \cdot v \in \Sigma^{n+m}$ definiert durch

$$(u \cdot v)(i) = \begin{cases} u(i) & 1 \leq i \leq n \\ v(i-n) & n < i \leq n+m \end{cases}$$

$\diamond$